A Lightweight Kernel-Native Emulation Framework for Transport Dynamics and Labeled Trace Generation in Heterogeneous Handovers

P. Papaioannou*, T. Politi†, C. Tranoris‡, S. Denazis*

*Electrical & Computer Eng. Dept., University of Patras, Greece,
†Department of Electrical and Computer Engineering (ECE) of the University of Peloponnese
‡p-Net, Patras Science Park
{papajohn, sdena}@upatras.gr, tpoliti@uop.gr, ctranoris@p-net.gr

***Abstract*—Heterogeneous access networks, including 5G, Wi-Fi, and emerging 6G technologies, require user equipment to transition between different access interfaces while maintaining active transport sessions. Evaluating transport-layer behavior under these conditions is difficult: discrete-event simulators abstract away real OS scheduling and socket-layer effects, while hardware testbeds are costly and hard to reproduce. This paper presents a lightweight, kernel-native emulation framework built on vanilla Linux network namespaces and tc netem, which executes unmodified application binaries over the native Linux TCP/IP stack while injecting controllable link impairments. The framework further incorporates a non-intrusive socket telemetry engine capable of 10-ms scale monitoring that logs kernel-level information such as congestion window, RTT, and retransmission metrics, producing labeled traces suitable for training machine learning models for traffic steering. We validate the framework through a case study of 100 MB file transfers, comparing an uninterrupted single-interface transfer against a transfer undergoing a vertical handover with active socket re-establishment (hard reset) at the 50% mark, under parametric sweeps of latency, jitter, and packet loss. Experimental results show that, under packet loss, active socket reset can reduce completion time compared with an uninterrupted connection. This behavior occurs because the new socket clears the RTO backoff state accumulated by the original connection.**

***Index Terms*—Network Emulation, Network Namespaces, TCP Telemetry, Handover, Socket Reset, Real-System Emulation Data**

I. Introduction

The deployment of 5G and emerging 6G networks is increasing the use of heterogeneous network architectures. Mobile devices may have multiple wireless interfaces, including cellular, Wi-Fi, and other access technologies. During mobility, an active data session may therefore need to transition between interfaces with different network characteristics. Such transitions can affect transport-layer state and performance.

During such handovers, connection performance may degrade because the characteristics of the active path can change. Reliable and repeatable tools are therefore needed to evaluate these transport-layer effects under controlled network conditions. These parameters include socket-level and kernel-level protocol behaviors under varied network impairments. Historically, researchers have chosen between discrete-event simulators (e.g., ns-3 [1]), lightweight container-based virtual emulators (e.g., Mininet [2]), full operating system virtualization using Virtual Machines (VMs), or hardware testbeds. While simulators are highly scalable, they rely on abstract mathematical approximations of the socket layer and TCP state machines, omitting real-world implementation issues. Conversely, hardware testbeds are expensive, difficult to configure, and hard to scale, while container-based systems can introduce virtualization overheads or timing deviations under heavy traffic emulation [3].

To bridge this gap, this paper introduces a lightweight, kernel-native network emulation framework built entirely on a vanilla Linux distribution. The primary objective of this framework is to provide a low-overhead, easily reproducible environment for testing, profiling, and validating various transport and application connection parameters over the Linux TCP/IP stack. By leveraging native kernel features—specifically network namespaces (`netns`) [4] and Traffic Control queueing disciplines (`tc netem`) [5], our framework executes unmodified, production-grade application binaries directly, capturing realistic system timing, CPU context-switching, and local OS scheduling behaviors.

Beyond running application-level tests, the framework is designed to generate data for offline analysis. It incorporates a non-intrusive, port-tracked socket telemetry engine that queries active socket states directly from the kernel using the Socket Statistics (`ss`) utility [6] at a millisecond scale. This enables the framework to output labeled semi-synthetic network traces that map real-world socket metrics (such as congestion window $cwnd$ [7], RTT, and retransmission statistics) to specific impairment labels, providing labeled traces that can be used for subsequent machine-learning analysis. In this work, we refer to the resulting traces as semi-synthetic because the application and transport metrics are obtained from real software implementations, while the network conditions are emulated.

We validate the utility of our framework through a transport-layer case study analyzing the vertical handover transition of a 100 MB file transfer during a vertical handover between two emulated network paths. We use the framework to perform automated multi-parameter sweeps (latency, loss, jitter, and rate limits) to evaluate the handover overhead of Active Socket

Re-establishment (intentionally resetting the connection state) relative to an uninterrupted single-link baseline transfer. The experiments also allow us to observe transport-layer behavior under controlled network impairments, including RTO backoff under packet loss and the effect of cached TCP metrics on newly established connections. For the PoC evaluation, identical impairment parameters are applied to both paths to isolate the effect of active socket re-establishment.

Problems Addressed

Despite significant advances in networking research, evaluating novel transport protocols, handover strategies, and application behaviors under dynamic network conditions remains challenging. This work addresses two critical challenges in modern network experimentation:

1) **The Simulation Fidelity vs. Resource Dilemma:** There is a lack of lightweight tools capable of executing unmodified application binaries directly on a host kernel while applying controlled latency and packet-loss profiles.
2) **Telemetry Distortion (The Observer Effect):** Capturing low level transient dynamics—such as the rapid throughput spikes during TCP Slow-Start—traditionally requires heavy packet-sniffing utilities (e.g., `tcpdump`). Under high-throughput or high-loss conditions, the CPU and disk I/O overhead of these tools alters system scheduling and distorts the underlying TCP/IP state machine behaviors being measured, obscuring true protocol performance.

To resolve these challenges, we introduce a lightweight network emulation platform. The key contributions of this work are summarized as follows:

- **A Versatile Network Emulator suitable for Digital Twins:** We design a lightweight, general-purpose network emulation framework using native Linux namespaces (`netns`) and Traffic Control (`tc`). Operating directly on the host kernel, this platform allows researchers to execute unmodified application-layer binaries (such as `lftp`) under precise, user-defined network impairments without the resource costs of virtual machines. Furthermore, the framework can be used as an emulation component in network Digital Twin environments.
- **Non-Intrusive Telemetry Tracking:** We introduce a low-overhead, polling-based telemetry trace mechanism that records socket progress with a minimal computing footprint. By avoiding heavy packet captures, our approach preserves system scheduling and captures fine-grained transient dynamics—including TCP Slow-Start bursts—without distorting the operational integrity of the stream.
- **Methodological Isolation and PoC Validation:** We document a rigorous experimental methodology (combining `tcp_no_metrics_save` and route cache flushes) that systematically eliminates kernel metric pollution across sequential runs. We validate the holistic framework using a hard IP-migration Proof of Concept (PoC), demonstrating its ability to capture and isolate highly dynamic, transient transport-layer transitions.

II. Related Work

The evaluation of transport protocol dynamics and connection parameter behaviors during vertical handovers has been investigated through various testing methodologies and protocol-level optimizations.

A. Network Emulation vs. Simulation

Evaluating transport protocols under dynamic network topologies typically relies on discrete-event simulators like ns-3 [1]. While simulators provide excellent scalability for massive topologies, they model the socket layer and TCP state machines using simplified mathematical approximations, omitting real-world OS scheduler overhead, context-switching latency, and driver-level queueing dynamics. Conversely, hardware testbeds are highly realistic but suffer from high costs, configuration complexity, and poor reproducibility. Software emulators like Dummynet [8] run in-kernel packet filtering to inject delays and drop rules, but they are often bound to specific operating system structures and lack container-level network isolation. Container-based virtual emulators like Mininet [2] execute real application binaries over the Linux kernel but are primarily designed for OpenFlow SDN architectures and can introduce higher CPU scheduling overheads under heavy traffic constraints [3]. In [9] the authors survey and compare various network emulators both commercial and open source while evaluating railroad communication networks. Recent network telemetry approaches have also used eBPF for low-overhead kernel monitoring [10]. However, eBPF architectures require complex program compilation, structural injection, and specific kernel privileges that reduce portable reproducibility across standard test environments. In contrast, our namespace-based testbed operates directly on the Linux networking stack and provides controlled link impairments while retaining the behavior of the host TCP/IP implementation.

B. Handover State Migration & Caching

To prevent performance degradation during handovers, early research explored state-caching and transport-layer feedback loops. Protocols like Freeze-TCP [11] attempt to prevent congestion window collapse during disconnections by advertising a zero window to the sender, freezing the connection state until the handover completes. Similarly, F-RTO (Forward RTO-Recovery) [12] detects spurious retransmission timeouts caused by temporary disconnections, preventing the sender from unnecessarily entering slow-start. At the socket layer, Snoeren and Balakrishnan [13] proposed an end-to-end connection migration architecture using TCP options to negotiate address handovers, allowing active sessions to persist across interface changes. However, this and other state-caching approaches rely on carrying over the Transmission Control Block (TCB) metrics (such as $cwnd$ and $ssthresh$) between paths. This carryover can be counterproductive if the new link's capacity is lower than the old path, or if the connection is trapped in a collapsed RTO backoff that throttles performance.

C. Modern Session Migration (MPTCP & QUIC)

To address path transitions natively, modern protocols integrate multi-homing. Multipath TCP (MPTCP) [14] enables a single connection to split traffic across multiple virtual interfaces using separate TCP subflows. In their empirical study, Paasch et al. [15] demonstrated that MPTCP can achieve seamless, "make-before-break" handovers between Wi-Fi and cellular networks by dynamically adding and removing subflows. However, they note that MPTCP subflows still suffer from state carryover. Similarly, QUIC [16] supports native connection migration by identifying connections via unique IDs rather than the 4-tuple, allowing active sessions to persist across IP address changes.

III. Emulation Testbed Architecture

The framework is designed to run on any standard Linux host or virtual machine without specialized hardware dependencies. It utilizes kernel namespaces to isolate the network environments of the client and server.

A. Namespace and Virtual Link Topology

The network topology is illustrated in Fig. 1.

1) **Isolated Namespaces:** We create a client namespace (‘left-ns‘) and a server namespace (‘right-ns‘). All routing tables, interface states, and socket connections are fully isolated.
2) **Dual Virtual Ethernet Paths:** We provision two independent virtual ethernet (‘veth‘) link pairs connecting the namespaces:
   - Link A: Connects ‘veth-left1‘ (Subnet 1: ‘10.0.1.1‘) to ‘veth-right1‘ (Subnet 1: ‘10.0.1.2‘).
   - Link B: Connects ‘veth-left2‘ (Subnet 2: ‘10.0.2.1‘) to ‘veth-right2‘ (Subnet 2: ‘10.0.2.2‘).

B. Dynamic Impairment Injection

Link characteristics are manipulated at the kernel queue level using Traffic Control (‘tc‘) with the Network Emulator (‘netem‘) qdisc. The framework supports dynamic, live impairment sweeps:

- **Bandwidth Rate Limiting:** Configured using standard ‘netem‘ rate parameters for our active evaluations (though Token Bucket Filter (‘tbf‘) is also supported by the emulation framework).
- **Latency & Jitter:** Injects propagation delays and normal/uniform delay distributions.
- **Packet Loss**: Emulates independent random loss probabilities (Bernoulli process) or multi-state correlation models (such as the 2-state Gilbert-Elliot or 4-state Markov chains supported by `netem`).

C. Handover Orchestration

During file transfer, a background control thread monitors the bytes transferred. Upon hitting a user-defined threshold (e.g., 50% progress), the scheduler triggers the handover:

1) **Link Invalidation:** Destroys the primary link (‘veth-right1‘ state set to ‘DOWN‘).
2) **Interface Promotion:** Activates the backup interface (‘veth-right2‘ state set to ‘UP‘).
3) **Destination Switching:** The client application is routed to target the alternative server IP address (‘10.0.2.2‘).

D. Software Harness and Orchestration Scripts

The emulation framework is orchestrated by a unified software harness composed of three primary scripts:

- **Core Emulation Script (`lftp-migration-test.sh`):** This bash script executes namespace lifecycle setup, interface configuration, virtual routing setup, and link degradation injection using `tc netem`. It spawns the FTP daemon and handles the vertical handover transition at the 50% download boundary by setting the active interface state to `DOWN`.
- **Socket Telemetry Daemon (`cwnd_logger.sh`):** This background logging script runs inside the server namespace. It continuously polls the kernel TCP socket metrics using the Linux `ss` utility at a high frequency, compiling real-time congestion window and RTT logs.
- **Automation Harness (`batch_runner.py`):** A Python orchestration script that parses configuration profiles from `config.json`. It automates parametric sweeps by executing iterations of the core test, managing cooldown states, and compiling raw trace iterations into a master dataset.

To facilitate reproducibility and encourage community extensions, the complete orchestration harness, dataset telemetry logs, and configuration sweeps are made publicly available at https://github.com/papajohn-uop/NetworkDynamicsDigitalTwin.git.

IV. Socket Telemetry & Labeled Trace Generation

To capture the exact transport-layer reaction to these handovers, the framework includes a non-intrusive socket telemetry engine.

A. Kernel-State Telemetry Logging

The logger operates in the background, executing in the sender's namespace (‘right-ns‘). It queries the active sockets via the Linux Socket Statistics (‘ss‘) tool:

$$\text{ss -t -i -n} \tag{1}$$

The ‘-n‘ flag is critical to prevent the OS from resolving numeric port boundaries (e.g., resolving the FTP control port ‘2121‘ to ‘iprop‘ in ‘/etc/services‘), ensuring consistent script parsing.

The telemetry stream is parsed using a lightweight ‘awk‘ engine that:

1) Filters out control channels by ignoring the configured control port (e.g., local port ‘2121‘ for FTP).
2) Detects the active interface by inspecting link liveness states, ignoring lingering sockets on dead links.
3) Extracts the exact congestion window (‘cwnd‘) segments, Round-Trip Time (RTT), and retransmission metrics.

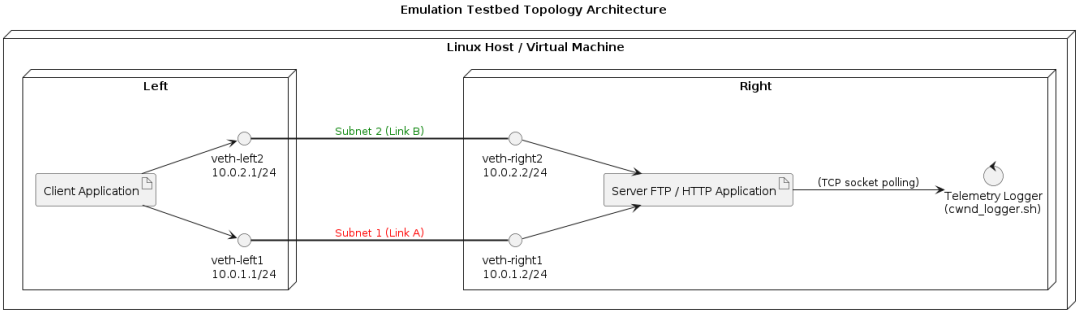

Fig. 1. Emulation testbed topology. A client namespace (`left-ns`) is connected to a server namespace (`right-ns`) via two independent virtual Ethernet (`veth`) interface paths simulating Subnet 1 and Subnet 2. Both paths are configured with controlled network parameters using `tc netem` queues.

B. Real-System Emulation Dataset Formulation

The telemetry data is appended to a structured CSV trace log along with high-precision epoch timestamps. Each row represents a labeled sample:

$$D = \{(t_i, S_i, c_i)\} \tag{2}$$

where t_i is the timestamp, S_i is the operational state label ('Phase1_Baseline', 'Phase2_Migrated', or 'Uninterrupted_Baseline'), and c_i is the congestion window size. By combining these traces with the configured link impairments (latency, loss, jitter) from the test configurations, the framework produces labeled datasets. The resulting traces can be used as input data for supervised or reinforcement-learning approaches to handover decision-making.

V. Case Study Evaluation: Handover Overhead of Active Socket Reset

To validate our emulation framework and demonstrate its utility in testing transport-layer parameters, we conduct a multi-parameter evaluation sweep analyzing a critical multi-connectivity handover scenario: Active Socket Re-establishment (hard reset) versus single link transfer, as baseline.

A. Experimental Setup

The emulation environment is provisioned on a bare-metal Linux host running Ubuntu 20.04 with a vanilla kernel. The system topology is configured as follows:

- **Application and Server:** We execute the standard `lftp` utility within the client namespace (`left-ns`) to download a 100 MB binary file populated with pseudorandom data. The server namespace (`right-ns`) runs an instance of a lightweight Python FTP daemon (`pyftpdlib`) bound to all interfaces.
- **Network Interfaces and Routing:** The namespaces are connected via two distinct subnets: Subnet 1 (Primary, `10.0.1.0/24`) and Subnet 2 (Backup, `10.0.2.0/24`). During the transfer baseline, traffic is routed through Subnet 1.
- **Impairment Scenarios:** Network impairments are injected at the egress interfaces of both namespaces using Linux `tc netem`. For each experimental configuration, the same impairment parameters are applied to both Subnet 1 and Subnet 2. Thus, although Subnet 1 is the primary path and Subnet 2 is the backup path, neither path is assumed to be inherently clean or degraded. The affected parameters include: Latency (10 ms to 40 ms), Independent Random Packet Loss (0.1% to 1.0%), and Latency Jitter (2 ms to 5 ms). The specific configurations are summarized in Table I.
- **Handover Orchestration:** Upon transferring exactly 50% of the file (50 MB), the background controller thread triggers the vertical handover. The primary link interface is set to `DOWN`, and the backup interface is promoted to `UP`. In the active reset strategy, the active socket is destroyed, and the client application immediately establishes a fresh TCP socket targeting Subnet 2 to resume the transfer. In the continuous baseline, the transfer is performed over the initial subnet with no switch,.
- **Data Collection:** The background telemetry daemon polls the kernel TCP metrics via `ss -t -i -n` at a 10 ms interval, outputting real-system emulation logs containing time-series traces of *cwnd*, RTT, and retransmissions. We execute 20 independent iterations per configuration to compile statistical summaries.

TABLE I
Emulation Test Configurations and Parametric Sweeps

Config ID	Sweep Domain	Bitrate , Latency , Jitter , Packet Loss
C1	Baseline	50 Mbps , 0 ms , 0 ms , 0%
C2	Latency	50 Mbps , 10 ms , 0 ms , 0%
C3	Latency	50 Mbps , 20 ms , 0 ms , 0%
C4	Packet Loss	50 Mbps , 20 ms , 0 ms , 0.1%
C5	Packet Loss	50 Mbps , 20 ms , 0 ms , 0.5%
C6	Packet Loss	50 Mbps , 20 ms , 0 ms , 1.0%
C7	Latency	50 Mbps , 40 ms , 0 ms , 0%
C8	Latency Jitter	50 Mbps , 40 ms , 2 ms , 0%
C9	Latency Jitter	50 Mbps , 40 ms , 5 ms , 0%

B. Handover Performance Sweeps

Table II aggregates the measured completion times and overhead statistics across the 20 iterations compiled for each configuration.

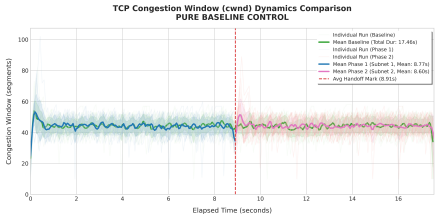

Fig. 2. Baseline Control (0ms, 0% Loss)

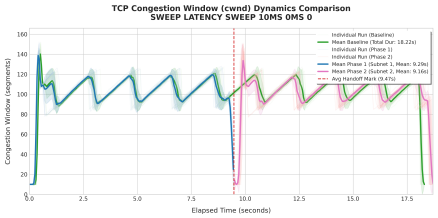

Fig. 3. Sawtooth Pattern (10ms, 0% Loss)

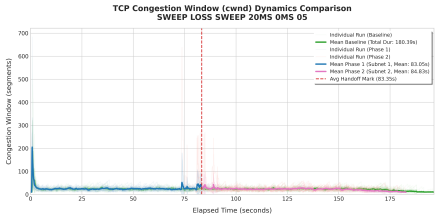

Fig. 4. Degraded Path (20ms, 0.5% Loss)

Fig. 5. Congestion window ($cwnd$) dynamics during vertical handovers, contrasting the clean control network (Fig. 2), the steady-state sawtooth latency profile (Fig. 3), and the degraded lossy link environment (Fig. 4).

TABLE II
HANDOVER PERFORMANCE AND COMPLETION TIME RESULTS (20 ITERATIONS PER CONFIG)

Config ID	Baseline μ (s)	Migration μ (s)	Overhead μ (s)
C1	17.602	17.657	0.054
C2	18.353	18.850	0.496
C3	19.552	20.619	1.067
C4	64.913	57.386	-7.528
C5	180.509	168.455	-12.053
C6	264.299	256.339	-7.961
C7	23.495	23.975	0.479
C8	22.396	24.229	1.833
C9	22.696	25.939	3.243

C. Analysis of the Crossover Threshold

As shown in Table II, the performance dynamics are highly dependent on the link conditions. To isolate the effect of socket re-establishment from changes in path quality, Subnet 1 and Subnet 2 are configured with the same network impairment parameters for each experiment. Thus, although the framework supports heterogeneous and asymmetric network conditions, the two paths in this PoC use identical latency, packet-loss, jitter, and rate-limit values. While real-world vertical handovers typically occur across highly asymmetric access networks (e.g., shifting from a degraded cellular link to a high-throughput WLAN), this configuration allows the observed differences between the baseline and active-reset transfers to be attributed primarily to the transport-layer state associated with the TCP connection. By holding the underlying physical environment constant across both interfaces, any measured delta in completion times or overhead cannot be attributed to shifting external link capacities. Instead, this symmetry isolates the internal transport-layer state machine behavior, providing a controlled baseline to evaluate pure connection transition policies—specifically, how state carryover flags stall protocol recovery versus how a hard socket reset forces an immediate escape from stale kernel timers.

The cwnd traces in Fig. 5 show three distinct behaviors.

1) Handover Phase Transitions: The time-series telemetry plots clearly delineate the handover transition stages:

- **Phase 1 (Baseline - Blue):** The download starts on Subnet 1. The socket enters standard TCP Slow Start, exponentially ramping up the congestion window to saturate the 50 Mbps path, eventually settling into CUBIC's congestion avoidance phase with a steady-state window of 150–270 segments.
- **The Transition Boundary (Vertical Red Dashed Line):** At precisely 50 MB (50% progress), the primary Subnet 1 interface is disabled.
- **Phase 2 (Migrated - Pink):** The client establishes a fresh TCP socket targeting the symmetric Subnet 2. As shown in the graphs, the $cwnd$ drops instantly to $W_{init} = 10$ segments—the standard Linux kernel initial congestion window (`initcwnd`). The socket then undergoes a fresh Slow Start phase on the backup subnet to complete the transfer.

2) RTT-Clocked Congestion Window Growth: The latency sweeps (10 ms, 20 ms, and 40 ms) illustrate the self-clocked nature of TCP congestion control. Because window expansion is driven by the arrival rate of ACKs, the rate of $cwnd$ growth is inversely proportional to the path's Round-Trip Time (RTT). Since Subnet 1 and Subnet 2 are homogeneously configured with the same delay, the $cwnd$ curve in Phase 2 scales with the same RTT-driven slope as Phase 1. Increasing RTT reduces the rate at which ACKs arrive and therefore slows cwnd growth during Slow Start.

3) Effect of RTO Backoff: Under lossy link conditions (0.1% to 1.0% packet loss), both Subnet 1 and Subnet 2 are subjected to the same packet loss rate. The results show that, under packet loss, the active-reset transfer can complete faster than the uninterrupted transfer, despite the additional handover operation (saving up to 12.0 seconds). The observed reduction in completion time is consistent with the RTO behavior of the continuing connection. Packet losses can trigger retransmissions and, when these retransmissions also fail, RTO backoff. The uninterrupted connection retains this state throughout the transfer, whereas establishing a new socket resets the retransmission timer and starts again with the initial congestion window. Under the tested loss conditions, the cost of restarting Slow Start is therefore lower than the accumulated RTO backoff of the uninterrupted connection.

4) Impact of Host Route Metrics Caching: While testing the framework, the behavior of the Linux routing cache metrics and the effect it has on the experiments was noted. By default, the Linux kernel saves TCP performance metrics (such as connection RTT and variance) in the routing table cache for reuse by subsequent connections to the same destination host [17]. During the Active Socket Reset transition, our telemetry monitored whether these cached metrics from the collapsed,

highly degraded Subnet 1 connection were inherited by the newly spawned socket on Subnet 2. We monitored the initial cwnd evolution after the socket was re-established on Subnet 2. The new connection started from the expected initial cwnd rather than the state observed on Subnet 1. This indicates that, under our experimental configuration, the previous connection state did not determine the initial congestion-window behavior of the new socket.

5) Jitter-Induced Spurious Fast Retransmits: The jitter sweeps (2 ms and 5 ms) demonstrate the impact of packet reordering under symmetric delay variance. Jitter causes packets to arrive out of order at the receiver, triggering duplicate ACKs. TCP mistakes these duplicate ACKs for packet loss, initiating spurious Fast Retransmits and unnecessarily halving the congestion window. The increased jitter results in a more pronounced saw-toothed pattern in the congestion window (cwnd), with repeated reductions followed by congestion-window growth. This behavior is consistent with the increased variation in packet delivery caused by jitter, which can lead to duplicate acknowledgments and retransmissions. As shown in Fig. 5 the higher-jitter configurations exhibit greater variation in the observed cwnd traces. The standard-deviation envelope is also wider, indicating increased variability in the congestion-window evolution during the transfer.

VI. Conclusion

In future work, we plan to expand this research along two primary axes. First, we intend to utilize the generated real-system emulation datasets to train deep reinforcement learning agents for predictive, intelligent traffic steering. Second, we will extend the orchestration framework to evaluate modern transport-layer protocols and alternative congestion control metrics. Specifically, we will compare the active socket reset policy against native Multipath TCP (MPTCP) subflows and QUIC connection migration [16] under asymmetric link degradations. This future iteration will place particular emphasis on analyzing how a hard socket reset interacts with rate-and-RTT-driven congestion control algorithms, such as Google's BBR [18], which fundamentally differ from loss-based algorithms like CUBIC in their architectural handling of packet loss and Retransmission Timeout (RTO) backoff states.

Data and Code Availability

The complete emulation testbed, traffic control (tc) configuration and telemetry scripts developed in this work are publicly available on GitHub at https://github.com/papajohn-uop/NetworkDynamicsDigitalTwin.git.

Acknowledgment

The authors' work has been supported by the HORIZON-JU-SNS Projects AMAZING-6G (GA No. 101192035) and SAND5G project (No 101127979), which received funding from the European Union's Digital Europe programme.

References

[1] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. Kopsel, "ns-3 project goals," Technical Report, ns-3 Project, Tech. Rep., 2008.

[2] B. Heller, R. Sherwood, and N. McKeown, "Openflow-based software-defined networks in mininet," in *Proceedings of the ACM SIGCOMM 2010 Conference on Posters and Demos*, 2010, pp. 3–4.

[3] L. Nussbaum and O. Richard, "A comparative study of network link emulators," in *Proceedings of the 12th Communications and Networking Simulation Symposium (CNS'09)*, San Diego, USA, 2009.

[4] E. W. Biederman, "Multiple instances of the global linux namespaces," in *Proceedings of the Linux Symposium*, vol. 1, 2006, pp. 101–112.

[5] S. Hemminger, "Network emulation with netem," in *Proceedings of the 6th Australian National Linux Conference (LCA2005)*, 2005.

[6] H. Stephen, "ss: Socket statistics utility for Linux," Part of the iproute2 package, https://git.kernel.org/pub/scm/network/iproute2/iproute2.git/, 2005.

[7] V. Jacobson, "Congestion avoidance and control," *ACM SIGCOMM Computer Communication Review*, vol. 18, no. 4, pp. 314–329, 1988.

[8] L. Rizzo, "Dummynet: a simple approach to the evaluation of network protocols," *ACM SIGCOMM Computer Communication Review*, vol. 27, no. 1, pp. 31–41, 1997.

[9] N. Fernández-Berrueta, J. Goya, J. Añorga, S. Arrizabalaga, G. De Miguel, and J. Mendizabal, "An overview of current ip network emulators for the validation of railways wireless communications," *IEEE Access*, vol. 8, pp. 109 266–109 274, 2020.

[10] C. Cassagnes, A. Tasiopoulos, and A. Mauthe, "An eBPF-based approach for low-overhead network telemetry," in *Proceedings of the IEEE International Conference on Cloud Networking (CloudNet)*, 2020, pp. 1–6.

[11] T. Goff, J. Moronski, D. Phatak, and V. Gupta, "Freeze-tcp: a true end-to-end tcp enhancement mechanism for mobile environments," in *Proceedings IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies (Cat. No.00CH37064)*, vol. 3, 2000, pp. 1537–1545 vol.3.

[12] P. Sarolahti, M. Kojo, K. Yamamoto, and M. Hata, "Forward rto-recovery (f-rto): An algorithm for detecting spurious retransmission timeouts with tcp," IETF, RFC 5682, 2009. [Online]. Available: https://tools.ietf.org/html/rfc5682

[13] A. C. Snoeren and H. Balakrishnan, "An end-to-end approach to host mobility," in *Proceedings of the 6th Annual International Conference on Mobile Computing and Networking (MobiCom '00)*. Boston, MA, USA: ACM, 2000, pp. 155–166.

[14] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, and C. Paasch, "Tcp extensions for multipath operation with multiple addresses," IETF, RFC 8684, 2020. [Online]. Available: https://tools.ietf.org/html/rfc8684

[15] C. Paasch, G. Detal, F. Duchene, C. Raiciu, and O. Bonaventure, "Exploring mobile/wifi handover with multipath tcp," in *Proceedings of the 2012 ACM SIGCOMM workshop on Cellular networks*, 2012, pp. 31–36.

[16] J. Iyengar and M. Thomson, "Quic: A udp-based multiplexed and secure transport," IETF, RFC 9000, 2021. [Online]. Available: https://tools.ietf.org/html/rfc9000

[17] J. Touch, "TCP Control Block Interdependence," RFC 9040, 2021. [Online]. Available: https://www.rfc-editor.org/info/rfc9040

[18] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *ACM Queue*, vol. 14, no. 5, pp. 20–53, 2016.